Step - 1: Study of given Example Codes

1. led_toggle.c

Analysis of led_toggle.c

- **Purpose:** This application continuously toggles an LED connected to a specific GPIO pin with a delay.
- **Pin Configuration: -** Port & Pin: GPIOD, Pin 12 (GPIO_PIN_NO_12).
- **Mode:** Output mode (GPIO_MODE_OUT).
- **Speed:** Fast speed (GPIO_SPEED_FAST).
- **Output Type:** Open-Drain (GPIO_OP_TYPE_OD).
- **Pull-up/Pull-down:** No pull-up, no pull-down (GPIO_NO_PUPD).

➢ **Driver APIs Used:**

- **GPIO_PeriClockControl():** Enables the peripheral clock for GPIOD.
- **GPIO_Init():** Initializes the LED pin with the specified configurations.
- **GPIO_ToggleOutputPin():** Toggles the state of the LED pin inside an infinite while (1) loop.

2. led_button.c

Analysis of led_button.c

- **Purpose:** This application reads the state of an external push button and toggles the LED when the button is pressed.
- **LED Configuration:** GPIOD, Pin 12 configured as an Output (GPIO_MODE_OUT), Push-Pull type (GPIO_OP_TYPE_PP), and Fast speed.
- **Button Configuration:** GPIOA, Pin 0 (GPIO_PIN_NO_0) configured as an Input mode (GPIO_MODE_IN) with Fast speed and no internal pull-up/pull-down.

➢ **Driver APIs Used:**

- **GPIO_PeriClockControl():** Enables peripheral clocks for both GPIOD and GPIOA.
- **GPIO_Init():** Initializes both the LED and Button handle structures.
- **GPIO_ReadFromInputPin():** Reads the digital value from GPIOA Pin 0 to check if the button is pressed (BTN_PRESSED).
- **GPIO_ToggleOutputPin():** Toggles the LED on GPIOD Pin 12 when the button press condition is met.

Board-Specific Considerations for STM32F401RE

- **Differences from STM32F446RE:** The provided code targets the STM32F446xx family, but if you are using the *STM32F401RE* (Nucleo-F401RE) board:
- **User LED Pin:** The onboard user LED (LD2) on the Nucleo-F401RE board is connected to *PA5* (GPIOA, Pin 5), whereas the example code uses *PD12* (GPIOD, Pin 12). You will need to change GPIOD to GPIOA and GPIO_PIN_NO_12 to GPIO_PIN_NO_5.
- **User Button Pin:** The onboard user push-button on the Nucleo-F401RE is typically connected to *PC13* (GPIOC, Pin 13). If you use the onboard button instead of an external one on PA0, you must update the button port to GPIOC and pin number to GPIO_PIN_NO_13.
- **Clock Enable:** Ensure the respective port clocks (RCC_AHB1ENR) match your updated GPIO ports (e.g., enabling GPIOC and GPIOA clocks).

3. gpio_interrupt.c

Analysis of gpio_interrupt.c

- **Purpose:** This application demonstrates external interrupt handling using a GPIO pin (button press) to toggle an LED.
- **LED Configuration:** GPIOD, Pin 12 (GPIO_PIN_NO_12) configured as an Output (GPIO_MODE_OUT) with low speed, push-pull type, and no pull-up/pull-down.
- **Button Configuration:** GPIOD, Pin 5 (GPIO_PIN_NO_5) configured as an External Interrupt Falling Trigger mode (GPIO_MODE_IT_FT) with fast speed and internal pull-up (GPIO_PIN_PU).

- **Driver APIs Used:**
  - **GPIO_PeriClockControl():** Enables the peripheral clock for GPIOD.
  - **GPIO_Init():** Initializes both the LED and Button structures.
  - **GPIO_IRQPriorityConfig() & GPIO_IRQInterruptConfig():** Configures the interrupt priority and enables the specific EXTI line interrupt in the NVIC.
  - **EXTI9_5_IRQHandler():** The Interrupt Service Routine (ISR) that clears the pending interrupt via GPIO_IRQHandling() and toggles the LED on GPIOD Pin 12.

4. spi_arduino_tx_test.c

Analysis of spi_arduino_tx_test.c

- **Purpose:** This application sets up the STM32 microcontroller as an SPI Master to transmit data (a string message) to an Arduino Slave.
- **Pins Used:** PB13 (SCLK), PB15 (MOSI), PB12 (NSS), configured in Alternate Function mode (GPIO_MODE_ALTFN, AF5) with fast speed and push-pull type.

- **Button Configuration:** GPIOA, Pin 0 (GPIO_PIN_NO_0) configured as an input to trigger the transmission on a button press.

➢ **Driver APIs Used:**

- **SPI2_GPIOInits() & SPI2_Inits():** Initialize the respective GPIO pins and configure SPI2 parameters (Full-Duplex Master mode, 8-bit DFF, SCLK speed divided by 32, hardware slave management disabled via SPI_SSM_DI).
- **SPI_SSOEConfig():** Enables NSS output control (SSOE) so the hardware automatically manages the NSS pin.
- **SPI_SendData():** Transmits the data length first followed by the actual string payload over SPI when the button is pressed.

Board-Specific Considerations for STM32F401RE

- **LED Pin Mapping:** The code uses GPIOD and GPIO_PIN_NO_12, whereas the Nucleo-F401RE board's user LED is connected to *PA5* (GPIOA, GPIO_PIN_NO_5). You must update the port and pin configurations accordingly.
- **Button Pin Mapping:** The interrupt code uses GPIOD Pin 5, and the SPI test uses GPIOA Pin 0. Ensure these matches your physical hardware wiring on the STM32F401RE board.

5. gpio_toggle.c

Analysis of gpio_toggle.c

- **Purpose:** To continuously toggle the state of an LED connected to a specific GPIO pin in an infinite loop using a simple delay-less transition (or continuous toggling).
- **Pin Configuration:** Configures GPIOA, Pin 5 (GPIO_PIN_NO_5) as a general-purpose output (GPIO_MODE_OUT).
- Sets output type to Push-Pull (GPIO_OP_TYPE_PP) with no pull-up/pull-down resistors (GPIO_NO_PUPD) and low speed (GPIO_SPEED_LOW).

➢ **Driver APIs Used:**

- **GPIO_PeriClockControl(GPIOA, ENABLE):** Enable the peripheral clock for GPIOA.
- **GPIO_Init(&GpioLed):** Apply the configuration settings.
- **GPIO_ToggleOutputPin(GPIOA, GPIO_PIN_NO_5):** Inside the while(1) loop to invert the pin's logic state continuously.

Board-Specific Considerations for STM32F401RE

- The user LED on the STM32F401RE Nucleo board is hardwired to *PA5* (GPIOA, Pin 5), which matches the configured pin.

6. led_ext_button.c

Analysis of led_ext_button.c

- **Purpose:** To control and toggle an LED (PA5) using an external push button connected to GPIOB Pin 12 (PB12) with a software delay-based debounce mechanism.
- **LED Pin:** GPIOA, Pin 5 (GPIO_PIN_NO_5) configured as an output (GPIO_MODE_OUT) with Push-Pull type (GPIO_OP_TYPE_PP), fast speed, and no pull-up/pull-down resistors.
- **External Button Pin:** GPIOB, Pin 12 (GPIO_PIN_NO_12) configured as an input (GPIO_MODE_IN) with an internal pull-up resistor enabled (GPIO_PIN_PU) and fast speed.

➢ **Driver APIs Used:**

- Enables peripheral clocks for both GPIOA and GPIOB using GPIO_PeriClockControl().
- Initializes both pin configurations using GPIO_Init().
- Continuously polls the input state of PB12 via GPIO_ReadFromInputPin().
- When the button press is detected (BTN_PRESSED == LOW), it executes a short debounce delay function and toggles the LED state using GPIO_ToggleOutputPin().

Board-Specific Considerations for STM32F401RE

- The onboard user LED is connected to *PA5*. Since the button is external, it uses PB12 with the internal pull-up resistor configuration activated.

7. spi_tx_test.c

Analysis of spi_tx_test.c

- **Purpose:** To initialize the SPI2 peripheral in Master mode (Full-Duplex) and transmit a test string buffer ("Hello world") to an external slave device using Software Slave Management (SSM).
- Configures GPIOB pins to alternate function mode (GPIO_MODE_ALTFN, AF5) with Push-Pull type, fast speed, and no pull-up/pull-down:
- **SCLK:** PB13 (GPIO_PIN_NO_13)
- **MOSI:** PB15 (GPIO_PIN_NO_15)

➢ **Note:** MISO (PB14) and NSS (PB12) are commented out in this test configuration since Software Slave Management handles NSS internally.

➢ **Driver APIs Used:**

- Configures SPI alternate function GPIOs through GPIO_Init().
- Sets up SPI2 parameters (Master mode, Full-Duplex bus config, 8-bit data frame format, CPOL_HIGH, CPHA_LOW, and SSM enabled) using SPI_Init().

- Pulls the internal NSS high using SPI_SSIConfig(SPI2, ENABLE) to prevent Mode Fault (MODF) errors.
- Enables SPI2 via SPI_PeripheralControl(), transmits data using SPI_SendData(), blocks until transmission completes by checking SPI_BUSY_FLAG, and disables the peripheral afterwards.

Board-Specific Considerations for STM32F401RE

- SPI2 alternate function mapping (PB13 for SCLK and PB15 for MOSI under AF5) matches the hardware layout guidelines of the STM32F401RE board.